

# LINUX TEMELLERİ

[Kaynak](#)

## İÇERİK

1. Linux Cheat Sheet
2. [10.1 Toolbox - Vim](#)
3. Linux Sistemlerde Dosya Transfer Yöntemleri ve Veri Paylaşımı
4. Linux Süreç Yönetimi (Process Management)
5. Sisteminizin Bakımı: Otomasyon (Cron Süreçleri ve Crontab Yönetimi)
6. Sistem Bakımı: Paket Yönetimi (Package Management)
7. Sistem Bakımı: Günlük (Log) Dosyaları ve Takibi
8. [Bash Scripting](#)
9. [CTF Analizi Karmaşık Flagleri Arama ve Regex Karakter Kümeleri](#)
10. Linux İzinleri 101: Dosya ve Kullanıcı Güvenliği

## Linux Cheat Sheet

[Linux Cheat](#)

## Linux Sistemlerde Dosya Transfer Yöntemleri ve Veri Paylaşımı

### 1. Web Üzerinden Dosya İndirme: `wget`

**wget**, dosyaları HTTP, HTTPS ve FTP gibi yaygın protokoller üzerinden indirmemizi sağlayan, terminal tabanlı oldukça temel bir araçtır. Tıpkı bir web tarayıcısının bir dosyaya erişmesi gibi çalışır.

- **Çalışma Mantığı:** Komutu takiben dosyanın bulunduğu tam web adresi (URL) belirtilir.
- **Kullanım Senaryosu:** Eğer "myfile.txt" adlı bir dosyanın web adresini biliyorsanız, bu dosyayı bulunduğunuz dizine indirmek için şu komut kullanılır: `wget https://assets.tryhackme.com/additional/linux-fundamentals/part3/myfile.txt`
- **Önemi:** Otomasyon süreçlerinde ve grafik arayüzü olmayan sunucularda internetten hızlıca kaynak çekmek için kritiktir.

### 2. Güvenli Dosya Transferi: SCP (Secure Copy)

**SCP (Secure Copy)**, dosyaları iki bilgisayar arasında güvenli bir şekilde kopyalamaya yarayan bir yöntemdir. Standart `cp` komutundan farkı, transfer sırasında veriyi hem

şifrelemesi hem de kimlik doğrulaması yapmasıdır. Bunu sağlamak için **SSH (Secure Shell)** protokolünü kullanır.

## Temel Özellikler

- **Kimlik Doğrulama:** İşlem yapabilmek için her iki sistemdeki kullanıcı adlarını ve şifrelerini (veya SSH anahtarlarını) bilmeniz gerekir.
- **Esneklik:** Yerel sistemden uzak sisteme veya uzak sistemden yerel sisteme çift yönlü kopyalama yapılabilir.

## Uygulama Senaryosu A: Yerel Makineden Uzak Sunucuya Gönderme

Aşağıdaki bilgilere sahip olduğumuzu varsayalım:

- **Uzak IP:** 192.168.1.30
- **Uzak Kullanıcı:** ubuntu
- **Yerel Dosya:** important.txt
- **Hedef İsim:** transferred.txt

**Komut Yapısı:** `scp important.txt`

`ubuntu@192.168.1.30:/home/ubuntu/transferred.txt` (Bu komut, kaynağı belirtilen uzak dizine, belirtilen yeni isimle kopyalar.)

## Uygulama Senaryosu B: Uzak Sunucudan Yerel Makineye Çekme

Senaryoyu tersine çevirelim: Uzak bir bilgisayarda ( 192.168.1.30 )

bulunan `documents.txt` dosyasını kendi makinemize `notes.txt` olarak kaydetmek isteyelim:

**Komut Yapısı:** `scp ubuntu@192.168.1.30:/home/ubuntu/documents.txt notes.txt`

## 3. Host Üzerinden Dosya Sunma: Web Sunucusu Oluşturma

Bazen kendi makinenizdeki bir dosyayı başkalarının (veya başka bir makinenin) indirmesini istersiniz. Ubuntu gibi sistemlerde yerleşik olarak gelen **Python3**, çok hızlı bir şekilde geçici bir web sunucusu kurmanıza olanak tanır.

### Python3 "HTTPServer" Modülü

Bu modül, komutun çalıştırıldığı dizini bir web sunucusu gibi yayına açar. Diğer kullanıcılar bu dosyalara `curl` veya `wget` ile erişebilir.

- **Sunucuyu Başlatma:** Terminalde şu komutu çalıştırmanız yeterlidir: `python3 -m http.server`
- **Varsayılan Ayarlar:** Sunucu genellikle **8000** portunda çalışır. Çalıştığı terminalde işlem sonlandırılana kadar aktif kalır.

## Uygulama Senaryosu: Dosya Sunma ve İndirme Süreci

- Hazırlık:** /webserver dizininde olduğunuzu ve içinde file isimli bir dosya olduğunu varsayalım. Sunucuyu başlatın: `python3 -m http.server` (Çıktı: *Serving HTTP on 0.0.0.0 port 8000 ...*)
- Dosyayı Çekme:** Başka bir terminal ekranı açarak (mevcut olan sunucuya ayrılmıştır) `wget` ile dosyayı indirin: `wget http://[MAKİNE_IP_ADRESİ]:8000/file`

## Dikkat Edilmesi Gereken Kısıtlamalar

- Port Belirtme:** Sunucu standart HTTP portu (80) yerine 8000'de çalıştığı için URL kısmında :8000 ibaresi mutlaka kullanılmalıdır.
- İndeksleme Sorunu:** Bu modülün en büyük zayıflığı otomatik bir dosya listesi (indeks) oluşturmamasıdır. İndirmek isteyen kişi, dosyanın tam adını ve konumunu bilmek zorundadır.
- Alternatifler:** Daha gelişmiş ve yine hafif bir sunucu arayanlar için **Updog** gibi araçlar önerilmektedir, ancak temel ihtiyaçlar için Python yerleşik modülü yeterlidir.

## Linux Süreç Yönetimi (Process Management)

Linux işletim sisteminde çalışan her program bir **Süreç (Process)** olarak adlandırılır. Bu süreçler doğrudan **Kernel (Çekirdek)** tarafından yönetilir.

### 1. Süreç Kimliği: PID Nedir?

Her sürecin kendine has bir kimlik numarası vardır ve buna **PID (Process ID)** denir.

- Artış Mantiği:** PID numaraları, süreçlerin başlama sırasına göre artarak devam eder. Örneğin; sistemdeki 60. süreç başlıyorsa, PID değeri 60 olur. Bir sonraki başlayan süreç 61 numarasını alır.
- Süreç Takibi:** PID sayesinde sistem, hangi programın ne kadar kaynak tükettiğini ve hangi kullanıcıya ait olduğunu ayırt eder.

### 2. Süreçleri Görüntüleme Komutları

Sistemde o an neyin çalıştığını görmek için iki temel komut kullanılır:

#### A. `ps` (Process Status) Komutu

Mevcut terminal oturumunuzda çalışan süreçleri listeler. Çıktıda sürecin durum kodu, CPU kullanım süresi ve çalışan programın adı gibi bilgiler yer alır.

- Geniş Kapsamlı Görüntüleme ( `ps aux` ):** Sadece kendi oturumunuzu değil, diğer kullanıcıların ve sistemin (root) yürüttüğü süreçleri görmek için kullanılır.
  - a:** Diğer kullanıcıların süreçlerini gösterir.
  - u:** Süreci başlatan kullanıcıyı ve detayları gösterir.

- **x:** Bir terminale bağılı olmayan (arka plan/sistem) süreçleri de listeler.

## B. top Komutu

Süreçleri tek seferlik bir liste yerine **gerçek zamanlı** olarak izlemenizi sağlar.

- **Yenileme:** İstatistikler varsayılan olarak her 10 saniyede bir güncellenir.
- **Etikleşim:** Ok tuşlarını kullanarak satırlar arasında gezinebilir ve sistem kaynaklarının anlık değişimini takip edebilirsiniz.

## 3. Süreçlerin Başlatılması ve Yapısı: Namespaces ve systemd

Linux, süreçleri yönetirken kaynakları (CPU, RAM vb.) paylaşmak için **Namespaces (İsim Alanları)** yapısını kullanır.

- **Namespace Mantığı:** Bilgisayar kaynaklarını bir pasta gibi dilimlere ayırır. Bir dilim içindeki süreçler sadece o dilimdeki güce erişebilir. Bu, **güvenlik ve izolasyon** sağlar; sadece aynı isim alanındaki süreçler birbirini görebilir.
- **Süreç Hiyerarşisi (Parent-Child):**
  - **PID 0:** Sistem önyükleme (boot) anında başlayan ilk süreçtir.
  - **systemd (init):** Ubuntu gibi sistemlerde kullanıcı süreçlerini yöneten, işletim sistemi ile kullanıcı arasında duran ana süreçtir.
  - **Child Process (Çocuk Süreç):** Başlattığımız her yazılım, **systemd** sürecinin bir "çocuğu" olarak doğar. **systemd** tarafından kontrol edilirler ancak kendi süreç kimlikleriyle bağımsız çalışırlar.

## 4. Süreçlerin Yönetimi ve Sinyaller

Çalışan bir süreci sonlandırmak veya duraklatmak için **Sinyaller** gönderilir. Bunun için en yaygın kullanılan araç **kill** komutudur.

- **Kullanım:** `kill [PID]` (Örn: `kill 1337`)
- **Önemli Sinyal Türleri:**
  1. **SIGTERM:** Süreci sonlandırır ancak kapanmadan önce "temizlik" (dosyaları kaydetme vb.) yapmasına izin verir. Kibar bir kapatma yöntemidir.
  2. **SIGKILL:** Süreci anında öldürür. Herhangi bir temizlik işlemine izin vermez; zorla kapatmadır.
  3. **SIGSTOP:** Süreci tamamen kapatmaz, sadece askıya alır/duraklatır.

## 5. Başlangıçta Çalışan Servisler: systemctl

Web sunucuları (Apache) veya veritabanları gibi kritik yazılımların sistem açıldığında otomatik başlaması istenir. Bu işlemleri yönetmek için **systemctl** komutu kullanılır.

- **Format:** `systemctl [seenek] [servis_adı]`
- **Temel Seenekler (Örnek: apache2 servisi üzerinden):**
  - **Start:** Servisi o an alıřtırır ( `systemctl start apache2` ).
  - **Stop:** alıřan servisi durdurur.
  - **Status:** Servisin o anki durumunu (alıřıyor mu, hata mı var) gösterir.
  - **Enable:** Servisi sistem her aıldığında otomatik bařlayacak řekilde ayarlar.
  - **Disable:** Otomatik bařlatma özelliğini iptal eder.

## 6. Ön Plan (Foreground) ve Arka Plan (Background) alıřma

Süreler terminalde iki farklı durumda alıřabilir:

### Arka Plan (Background) Kullanımı

Bir komutun sonuna `&` operatörü eklendiğinde, süre arka planda alıřır.

- **Avantajı:** Özellikle uzun süren dosya kopyalama veya büyük betik (script) işlemlerinde, işlemin bitmesini beklemeden terminali kullanmaya devam etmenizi saęlar.
- **Örnek:** `echo "Merhaba" &` komutunu alıřtırdığınızda ıktı terminale düşmek yerine size sadece sürecin ID'sini verir.

### Süreci Askıya Alma ve Geri Getirme

1. **Ctrl + Z:** Ön planda alıřan bir betięi duraklatır ve arka plana atar. Terminaldeki mesaj kalabalığını anında kesmek için idealdir.
2. **fg Komutu (Foreground):** Arka plana atılmış veya askıya alınmış bir süreci tekrar ön plana, yani etkileřim kurabileceğiniz odak noktasına getirir.

## Sistem Bakımı: Otomasyon (Cron Süreleri ve Crontab Yönetimi)

Sistem önyükleme (boot) yapıldıktan sonra belirli görevlerin otomatik olarak yürütölmesi, sistem yönetimi ve verimlilik için kritiktir. Bu notlarda, Linux/Unix tabanlı sistemlerde görev zamanlamanın temeli olan **Cron** yapısını inceleyeceğiz.

### 1. Cron ve Crontab Nedir?

- **Cron Süreci:** İşletim sistemi bařlatıldığında (boot) devreye giren ve arka planda sürekli alıřan bir servistir. Temel sorumluluęu, zamanlanmış görevleri (cron jobs) yönetmek ve doęru zamanda tetiklemektir.
- **Crontab (Cron Table):** Cron sürecinin okuyup işleyebileceęi, özel bir biçimlendirmeye sahip olan dosyadır. Bu dosya, sistemin her satırı adım adım işlemlerini saęlayan bir talimat listesi görevi görür.

### 2. Crontab Dosya Yapısı ve Deęerler

Bir crontab satırı, görevin ne zaman ve nasıl çalışacağını belirleyen **6 spesifik değerden** oluşur. Bu değerlerin sırası ve anlamı şöyledir:

| Değer | Açıklama                                                                 |
|-------|--------------------------------------------------------------------------|
| MIN   | Görevin yürütüleceği <b>Dakika</b> (0 - 59)                              |
| HOURL | Görevin yürütüleceği <b>Saat</b> (0 - 23)                                |
| DOM   | Ayın hangi <b>Günü</b> (Day of Month) (1 - 31)                           |
| MON   | Yılın hangi <b>Ay</b> (Month) (1 - 12)                                   |
| DOW   | Haftanın hangi <b>Günü</b> (Day of Week) (0 - 6, genellikle 0 Pazar'dır) |
| CMD   | Çalıştırılacak olan asıl <b>Komut</b>                                    |

### 3. Joker Karakter (Wildcard) Kullanımı

Crontab içerisinde kullanılan en önemli sembol **Asterisk (\*)** yani yıldız işaretidir.

- **Mantığı:** Eğer belirli bir alan için (ay, gün veya yıl fark etmeksizin) özel bir kısıtlama getirmek istemiyorsanız yıldız kullanırsınız.
- **Örnek:** "Hangi ay olduğu önemli değil, her ay çalışsın" demek için **MON** alanına **\*** konulur.

### 4. Uygulama Senaryosu: Dosya Yedekleme

Bir sistem yöneticisinin "cmnatic" isimli kullanıcının "Documents" klasörünü **her 12 saatte bir** yedeklemek istediğini varsayalım. Bu senaryonun yapılandırılması şu şekildedir:

#### Komut Yazımı:

```
0 */12 * * * cp -R /home/cmnatic/Documents /var/backups/
```

- **Neden 0 ?:** Saatin ilk dakikasında başlamasını sağlar.
- **Neden \*/12 ?:** Bölü işareti ile kullanılan bu ifade, "her 12 saatte bir" tekrar etmesi gerektiğini belirtir.
- **Neden cp -R ?:** Klasörün içeriğini (recursive) hedef klasöre kopyalamak için kullanılan asıl komuttur.

### 5. Crontab Düzenleme ve Yardımcı Araçlar

Crontab dosyaları doğrudan metin editörüyle açılıp düzenlenmez; sistemin değişikliği fark etmesi için özel komutlar kullanılır:

- **crontab -e Komutu:** Mevcut kullanıcının crontab dosyasını düzenlemek için kullanılır. Bu komut çalıştırıldığında sistem size bir editör (genellikle **Nano** veya **Vi**) seçtirir ve dosya üzerinde değişiklik yapmanıza olanak tanır.

- **Önemli Kaynaklar:** Crontab sözdizimi başlangıçta karmaşık gelebilir. Bu süreci kolaylaştırmak için aşağıdaki araçlar kullanılır:
  - **Crontab Generator:** Kullanıcı dostu bir arayüzle seçim yapmanızı sağlayıp size otomatik komut satırı üretir.
  - **Cron Guru:** Yazdığınız cron ifadesinin tam olarak hangi zaman denk geldiğini insan dilinde açıklayan (örneğin: "Her Salı saat 04:00'te") bir izleme aracıdır.

## Sistem Bakımı: Paket Yönetimi (Package Management)

İşletim sisteminin güncel kalması ve yeni yetenekler kazanması, paket yönetimi süreçlerinin doğru işletilmesine bağlıdır. Bu bölümde Linux'un açık kaynak gücünü yansıtan **apt** yapısını ve üçüncü taraf yazılım ekleme süreçlerini inceleyeceğiz.

### 1. Paketler ve Yazılım Depolarına (Repositories) Giriş

- **Yazılım Dağıtım Süreci:** Geliştiriciler, hazırladıkları yazılımları topluluğa sunmak için bir **"apt" deposuna** gönderirler. Yazılım onaylandığında genel kullanıma sunulur. Bu yapı, Linux'un kullanıcı erişilebilirliği ve açık kaynak liyakati ilkelerinin temelini oluşturur.
- **Depo Türleri:** İşletim sistemi üreticileri kendi resmi depolarını barındırır da, kullanıcılar kendi listelerine **topluluk depolarını** ekleyebilirler. Bu, işletim sisteminin yeteneklerini varsayılanın ötesine taşır.
- **Bölgesel Depolar:** Bazı sağlayıcılar, indirme hızını artırmak amacıyla kullanıcıya coğrafi olarak daha yakın olan yerel depolar sunar.

### 2. APT (Advanced Package Tool) Ekosistemi

Ubuntu sistemlerde yazılım yönetimi için temel araç **apt** komutudur.

- **İşlevsellik:** Apt, sadece yazılım yüklemekle kalmaz; paket kaynaklarını yöneten, yazılımları güncelleyen veya sistemden kaldıran bir araçlar paketidir.
- **Avantajları:** **dpkg** gibi bağımsız paket yükleyicilerin aksine **apt** kullanımı, sistem güncellendiğinde eklenen tüm depoların otomatik olarak kontrol edilmesini sağlar. Böylece yazılımlar her zaman güncel kalır.

### 3. Depo Yönetimi ve Güvenlik: GPG Anahtarları

Yeni bir yazılım deposu eklenirken bütünlük ve güvenlik **GPG (Gnu Privacy Guard)** anahtarları ile sağlanır.

- **GPG Mantiği:** Bu anahtarlar, geliştiricilerin "Bu yazılım gerçekten bizim tarafımızdan hazırlandı" dediği dijital bir güvenlik kontrolüdür.
- **Eşleşme Kuralı:** Eğer geliştiricinin anahtarı ile sistemin güvendiği anahtarlar eşleşmezse, güvenlik gerekçesiyle yazılım indirilmez.

## 4. Uygulama Senaryosu: Sublime Text 3 Kurulumu (Manuel Yöntem)

Varsayılan Ubuntu depolarında bulunmayan **Sublime Text** editörünü sisteme eklemek için izlenen adım adım süreç:

### Adım 1: GPG Anahtarının Güven Listesine Eklenmesi

İlk olarak geliştiricinin dijital imzası indirilir ve sisteme tanıtılır: `wget -q0 - https://download.sublimetext.com/sublimehq-pub.gpg | sudo apt-key add -`

### Adım 2: Depo Dosyasının Oluşturulması

En iyi uygulama (best practice), her üçüncü taraf depo için ayrı bir dosya oluşturmaktır.

- `/etc/apt/sources.list.d` dizini altında `sublime-text.list` adında bir dosya oluşturulur.
- İlgili depo adresi bir metin editörü (Nano vb.) yardımıyla bu dosyaya yazılır ve kaydedilir.

### Adım 3: Kayıt Defterinin Güncellenmesi

Yeni eklenen deponun sistem tarafından tanınması için apt indeksleri güncellenmelidir: `sudo apt update`

### Adım 4: Kurulumun Tamamlanması

Artık sistem yazılıma güvendiği ve nerede bulacağını bildiği için kurulum yapılır: `sudo apt install sublime-text`

## 5. Paketlerin ve Depoların Kaldırılması

Yükleme sürecinin tersine çevrilmesi oldukça basittir:

- **Depoyu Kaldırma:** \* Yöntem A: `add-apt-repository --remove ppa:PPA_Adı/ppa` komutu kullanılır.
  - Yöntem B: `/etc/apt/sources.list.d` içinde manuel oluşturulan dosya silinir.
- **Yazılımı Kaldırma:** Depo kaldırıldıktan veya devre dışı bırakıldıktan sonra yazılımın kendisi şu komutla silinir: `sudo apt remove sublime-text`

## Sistem Bakımı: Günlük (Log) Dosyaları ve Takibi

Sistem günlükleri, bir işletim sisteminin "hafızası" gibidir. Çalışan uygulamalar, servisler ve sistemin kendi sağlığı hakkında kritik veriler sunar.

### 1. Log Dosyalarının Konumu ve Yönetimi

- **Dizin Yapısı:** Linux sistemlerde tüm günlük dosyaları ve klasörleri standart olarak `/var/log` dizini altında toplanır.

- **Log Rotasyonu (Rotating):** İşletim sistemleri, günlük dosyalarının diskte çok fazla yer kaplamasını önlemek için bu dosyaları otomatik olarak yönetir. Bu süreç "rotasyon" denir; eski kayıtlar sıkıştırılır veya belirli bir süre sonra silinerek sistemin depolama sağlığı korunur.

## 2. Kritik Servisler ve Log Örnekleri

Sistemin sağlığını izlemek ve güvenliğini korumak için özellikle üç temel servisin günlükleri hayati önem taşır:

1. **Apache2 (Web Sunucusu):** Bir web sunucusuna gelen her bir isteği kaydeder.
2. **Fail2ban:** Sistemdeki "brute force" (kaba kuvvet) saldırı girişimlerini izlemek ve engellemek için kullanılır.
3. **UFW (Uncomplicated Firewall):** Güvenlik duvarı (firewall) kayıtlarını tutarak ağ trafiğini izlemeye yardımcı olur.

## 3. Web Sunucusu Günlüklerinin Analizi

Bir web sunucusu (örneğin Apache2) özelinde, geliştiriciler ve yöneticiler için iki temel dosya türü performans ve güvenlik teşhisi için kullanılır:

- **Access Log (Erişim Günlüğü):** Sunucuya gelen her bir isteğin (request) bilgisini içerir. Kimin, ne zaman, hangi sayfaya erişmeye çalıştığını gösterir.
- **Error Log (Hata Günlüğü):** Sunucuda meydana gelen hataları, yanlış yapılandırmaları veya çalışmayan betikleri listeler.

**Analiz Neden Önemlidir?** Bu dosyalar sadece performans sorunlarını teşhis etmek için değil, aynı zamanda sisteme sızmaya çalışan bir saldırganın (intruder) ayak izlerini ve faaliyetlerini takip etmek için de kullanılır.

## 4. İşletim Sistemi ve Kullanıcı Kayıtları

Sadece uygulamalar değil, işletim sisteminin kendisi de operasyonel verileri kayıt altına alır:

- **Sistem Çalışma Kayıtları:** İşletim sisteminin arka planda nasıl çalıştığına dair teknik detaylar.
- **Kullanıcı Eylemleri:** Kullanıcıların sistem üzerinde gerçekleştirdiği işlemler.
- **Kimlik Doğrulama (Authentication) Girişimleri:** Sisteme giriş denemeleri, başarılı/başarısız oturum açma kayıtları bu kategoride yer alır. Bu kayıtlar, yetkisiz erişim denemelerini tespit etmek için ilk bakılacak yerdir.

## Linux İzinleri 101: Dosya ve Kullanıcı Güvenliği

Linux sistemlerinde her dosya ve dizin, "kimin ne yapabileceğini" belirleyen sıkı bir izin mekanizmasına tabidir. Sızma testlerinde (Pentest) en büyük amacımız genellikle bu izinleri aşmak (**Privilege Escalation**) veya yanlış yapılandırılmış izinleri sömürmektir.

# 1. İzinleri Okumak: `ls -lh` Analizi

Bir dizindeki dosyaların izinlerini detaylıca görmek için `ls -lh` komutunu kullanırız. Çıktının ilk üç sütunu bizim için hayati önem taşır:

Bash

```
tryhackme@linux2:~$ ls -lh
-rw-r--r-- 1 cmnatic cmnatic 0 Feb 19 10:37 file1
```

## Sütunların Anlamı:

- `-rw-r--r--` : İzinler (Permissions). İlk karakter dosya tipidir ( `-` dosya, `d` dizin). Sonraki 9 karakter ise izinleri temsil eder.
- `cmnatic` (**İlk isim**): Dosyanın sahibi olan **kullanıcı (Owner)**.
- `cmnatic` (**İkinci isim**): Dosyaya erişimi olan **grup (Group)**.

## İzinlerin Mantığı (Read, Write, Execute)

- Read (r)**: Dosyayı okuma/görüntüleme izni.
- Write (w)**: Dosyayı düzenleme/silme izni.
- Execute (x)**: Dosyayı bir program veya script olarak çalıştırma izni.

# 2. Kullanıcılar ve Gruplar: Neden Ayrımlar Var?

Linux'ta izinler çok hassas (**granular**) şekilde ayarlanabilir. Bir dosyanın sahibi siz olabilirsiniz, ancak bir grup kullanıcıya da o dosya üzerinde işlem yapma yetkisi verebilirsiniz.

- Gerçek Hayat Senaryosu**: Bir web sunucusunu (Apache/Nginx) çalıştıran bir sistem kullanıcısı vardır. Bu kullanıcının web dosyalarını okuması/yazması gerekir. Ancak, web hosting müşterilerinin de kendi dosyalarını yükleyebilmesi gerekir. Eğer hepsi aynı "root" veya "web" kullanıcısı olsaydı, bir müşterinin hatası tüm sistemi tehlikeye atardı. Gruplar ve ayrı kullanıcılar sayesinde bu risk izole edilir.

# 3. Kullanıcı Değiştirme: `su` Komutu

Bir kullanıcıdan diğerine geçmek için `su` (**Substitute User**) komutunu kullanırız. Bunun için hedef kullanıcının parolasını bilmemiz gerekir.

- `su user2` : Klasik geçiş. Sizi yeni kullanıcıya geçirir ama hala eski kullanıcının bulunduğu dizinde kalırsınız.
- `su -l user2` (veya `su - user2`) : Bu yöntem en sağlıklısidir. Hedef kullanıcının **environment variables** (ortam değişkenlerini) ve profil özelliklerini miras alırsınız. Sanki o kullanıcıyla bilgisayarı yeni açmış gibi doğrudan onun `home` dizinine gidirsiniz.

## Bash

```
# Örnek: Login parametresi ile geçiş
tryhackme@linux2:~$ su -l user2
Password:
user2@linux2:~$ pwd
/home/user2
```

## 4. Sayısal İzin Formatı (Octal Notation)

İzinler bazen `rwxr-xr-x` gibi harflerle, bazen de `755` gibi sayılarla ifade edilir. Bu sayıların her biri, üçlü grupların toplam değeridir.

### Hesaplama Tablosu

Her iznin bir sayısal karşılığı vardır:

| İzin        | Değeri |
|-------------|--------|
| Read (r)    | 4      |
| Write (w)   | 2      |
| Execute (x) | 1      |

### Gruplandırma ve Toplama

İzinler her zaman üçlü gruplar halinde hesaplanır: **Owner (Sahip) - Group (Grup) - Others (Diğerleri)**.

**Örnek:** `rwxr-xr-x`

- Sahip (rwx):** 2 =
- Grup (r-x):** =
- Diğerleri (r-x):** =

- Sonuç:** 755

### Sık Karşılaşılan Örnekler

| Sembolik               | Sayısal    | Açıklama                                                        |
|------------------------|------------|-----------------------------------------------------------------|
| <code>rwXrwxrwx</code> | <b>777</b> | Herkes her şeyi yapabilir. ( <b>Büyük Güvenlik Riski!</b> )     |
| <code>rwXr-xr-x</code> | <b>755</b> | Sahibi her şeyi yapar, diğerleri sadece okur ve çalıştırır.     |
| <code>rw-r--r--</code> | <b>644</b> | Sahibi okur/yazar, diğerleri sadece okur. (Standart dosya izni) |
| <code>rwX-----</code>  | <b>700</b> | Sahibi dışında hiç kimse hiçbir şey yapamaz.                    |

## 5. İzinleri Değiştirmek: `chmod`

Dosya izinlerini değiştirmek için `chmod` komutu kullanılır.

**Örnek:** `chmod 750 sistem_ozeti.txt`

- **Sahibi (7):** Tam yetki.
- **Grup (5):** Okuyabilir ve çalıştırabilir.
- **Diğerleri (0):** Hiçbir yetkisi yok (dosyayı göremezler bile).

**Neden Önemli?** Pentest yaparken, ele geçirdiğin bir dizinde hassas dosyaların (örneğin `.ssh` klasörü veya `config.php`) izinlerini kontrol etmelisin. Eğer `644` olması gereken bir dosya `777` ise, bu senin için bir sızma noktası olabilir.